

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been copied and communicated to you by or on behalf of The University of Newcastle pursuant to Part VB of the Copyright Act 1968 (the Act).

The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

SENG1050

Web Technologies

Week 5 - Advanced HTML and CSS

Lecture Outline

- ☐ HTML Structural Elements
- ☐ HTML Forms
- ☐ HTML Tables
- ☐ CSS Reset
- ☐ CSS Layout

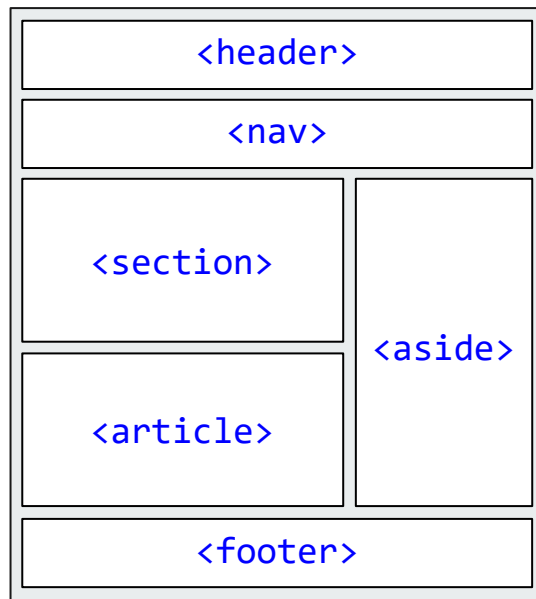
Recall from last week: CSS

- ❑ We have learned to separate the structure and semantics of our website from the data and the presentation components.
- ❑ We write the structure and semantics in HTML, the data in XML, and the presentation in CSS.
- ❑ Aside from just separating components, using CSS allows us to override browser styling to provide a consistent presentation across different browsers.
- ❑ But there is a problem with this: the styles we don't specify still use browser defaults, potentially leading to visual inconsistencies across browsers.
- ❑ Additionally, for simplicity, we skipped many important features of HTML and CSS in the beginning, to create elements such as forms, tables, and custom page layouts.
- ❑ Today, we will use what we have already learned about HTML and CSS to extend our toolbox to include these elements and finish the separation of structure and presentation.

HTML Structural Elements

HTML Structural Elements

- ❏ AKA: Semantic structural elements
- ❏ HTML defines some semantic elements that can be used to define different parts of a web page.
- ❏ Not intended to replace **div** completely rather to use to structure the web page in a more meaningful manner



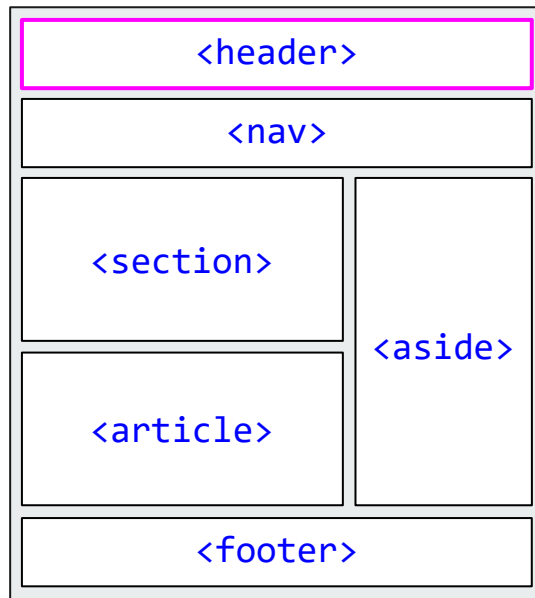
HTML Structural Elements

- ❑ Header Element: `<header>`
- ❑ Nav Element: `<nav>`
- ❑ Footer Element `<footer>`
- ❑ Main Element: `<main>`
- ❑ Aside Element: `<aside>`
- ❑ Section Element: `<section>`
- ❑ Article Element: `<article>`

HTML Structural Elements

Header Element: `<header>`

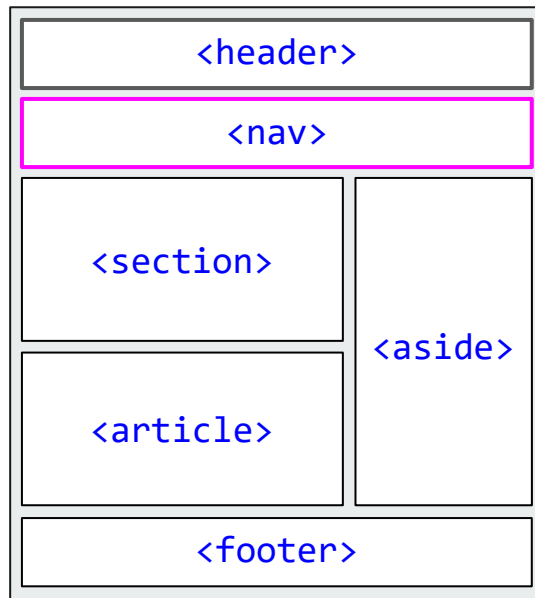
- ❑ Contains the headings of
 - a web page document, or
 - an area in the document such as a section or article
- ❑ Should be used as a container for introductory content or set of navigational links.
- ❑ You can have several `<header>` elements in one document.
- ❑ But cannot be placed within a `<footer>`, `<address>` or another `<header>` element.



HTML Structural Elements

Nav Element: `<nav>`

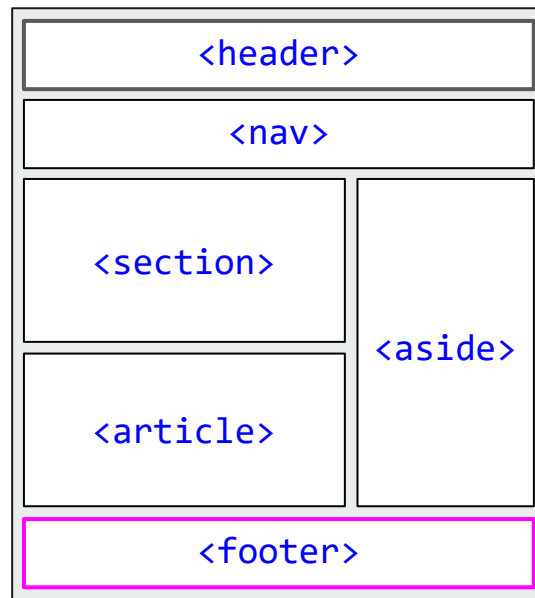
- ❑ Defines a block of navigation links
- ❑ Contains a section of navigation hyperlinks
- ❑ Note that NOT all links of a document should be inside a `<nav>` element.
- ❑ The `<nav>` element is intended only for **major block of navigation links.**
- ❑ Should not be placed within a footer to include links in the footer.



HTML Structural Elements

Footer Element: `<footer>`

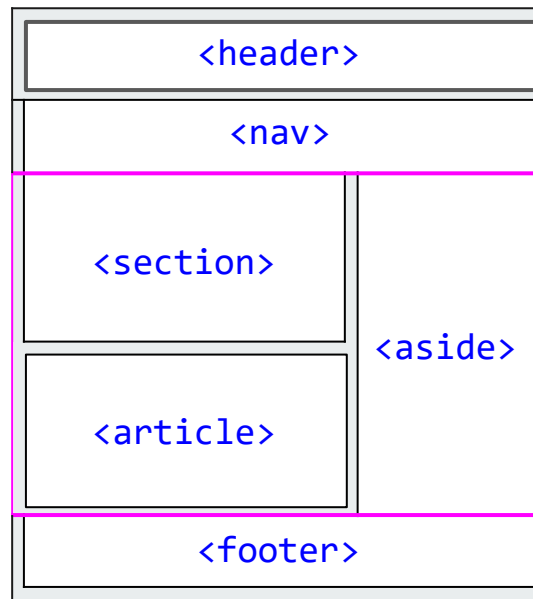
- ❑ Defines a footer for a document or section
- ❑ Can contain information such as:
 - Authorship
 - Copyright
 - Contact info
 - Back to top links
 - Related documents, etc.
- ❑ You can have several `<footer>` elements in one document.
- ❑ Cannot be inside of another `<footer>` or `<header>` element.



HTML Structural Elements

Main Element: `<main>`

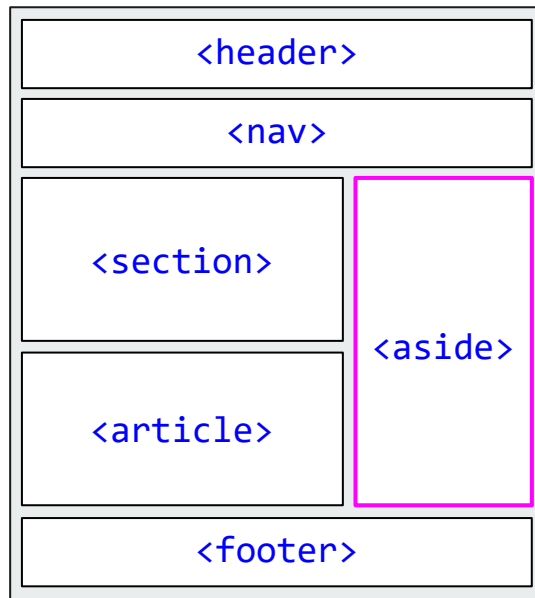
- ❑ Contains main page content
- ❑ The content inside the `<main>` element should be unique to the document.
- ❑ There must not be more than one visible `<main>` element in a document.
- ❑ The `<main>` element can only be a descendent of the `<html>`, `<body>`, `<div>`, or `<form>` elements.



HTML Structural Elements

Aside Element: `<aside>`

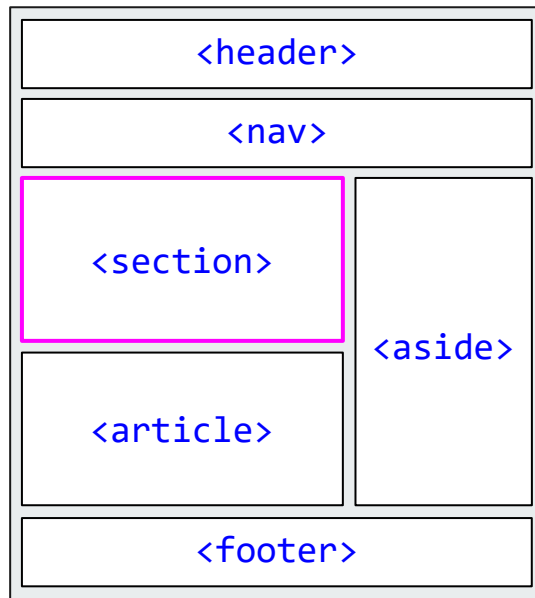
- ❑ Contains a sidebar, a note, or other tangential content
- ❑ Defines some content aside from the content it is placed in.
- ❑ The aside content should be related to the surrounding content.
 - Enhance an article with additional information.
 - Highlight parts that can be interesting



HTML Structural Elements

Section Element: `<section>`

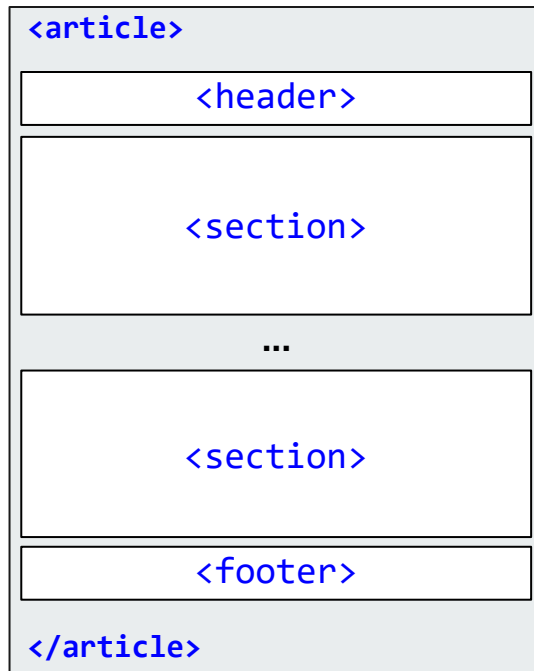
- ❑ contains a “section” of a document, such as a chapter or topic or any other sections of the document
 - **W3C:** “A section is a thematic grouping of content, typically with a heading.”
- ❑ A web page could normally have multiple sections, such as introduction, content, conclusion, etc.



HTML Structural Elements

Article Element: `<article>`

- ❑ Contains an independent entry
 - Forum post
 - Blog post
 - Newspaper article
- ❑ Should be possible to distribute it independently from the rest of the site.
- ❑ It is possible for HTML pages to have `<section>` elements containing `<article>` elements, or `<article>` elements containing `<section>` elements.



```

<body>
<section>
  <header>
    <h1>Big Header: HTML5 Semantic Structure</h1>
  </header>

  <nav>
    <a href="#home"> home </a>
    <a href="#about"> about </a>
  </nav>

  <main>

    <aside style="float:right; width:20%">
      <h2> aside </h2>
      <p> The aside element is used to represent content that is tangibly
        related to the content surrounding it, but could be considered
        separate.</p>
    </aside>

    <section>
      <article>
        <header>
          <h2> article 1 header SENG1050 </h2>
        </header>
        <section>
          <h3> section 1 </h3>
          <p> This is the first section.</p>
        </section>
        <footer>
          article 1 footer SENG1050
        </footer>
      </article>
    </section>

    <footer>
      &copy; University of Newcastle 2021
    </footer>

  </main>
</body>

```

Big Header: HTML5 Semantic Structure

[home](#) [about](#)

article 1 header SENG1050

section 1

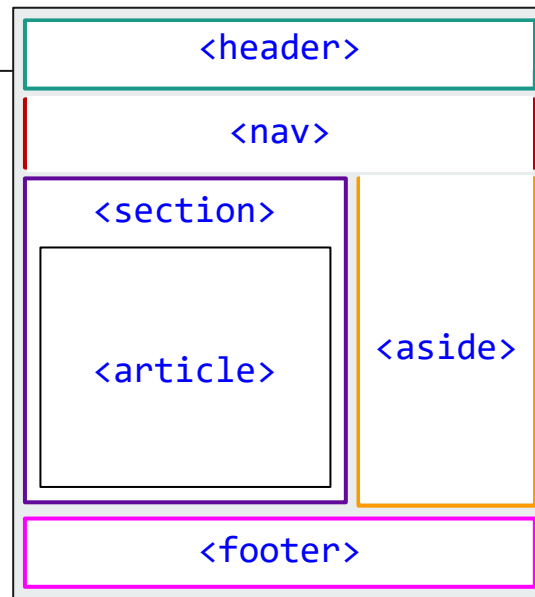
This is the first section.

article 1 footer SENG1050

© University of Newcastle 2021

aside

The aside element is used to represent content that is tangibly related to the content surrounding it, but could be considered separate.



<figure>, <figcaption> Tags

```
<figure>
  
  <figcaption>
    Fig. 1 - A view of the <mark>pulpit rock</mark> in Norway.
  </figcaption>
</figure>
```



Fig. 1 - A view of the **pulpit rock** in Norway.

<details>, <summary> Tags

```
<details>
  <summary>Copyright 1999-2014.</summary>
  <p> - by Refsnes Data. All Rights Reserved.</p>
  <p>
    All content and graphics on this
    web site are the property of the company Refsnes Data.
  </p>
</details>
```

Show/Hide

► Copyright 1999-2014.

▼ Copyright 1999-2014.

- by Refsnes Data. All Rights Reserved.

All content and graphics on this web site are the property of the company Refsnes Data.

<progress> Tag

Downloading progress:

```
<progress value="22" max="100">  
</progress>
```

```
<p>Finishing at <time>10:00</time>.</p>
```

Can be
updated
with
Javascript

Downloading progress:



Finishing at 10:00.

<audio> Tag

- ❏ The `<audio>` tag defines sound, such as music or other audio streams.
- ❏ Supported file formats for the `<audio>` element include
 - MP3, WAV, OGG, MP4 (MP3 is most commonly used)
 - Not all formats are supported by all browsers
 - **controls** indicates that audio controls should be displayed (such as play/pause buttons)

```
<audio controls>  
  <source src="horse.ogg" type="audio/ogg">  
  <source src="horse.mp3" type="audio/mpeg">  
  Your browser does not support the audio element.  
</audio>
```



<video> Tag

- ❑ The **<video>** tag specifies video, such as a movie clip or other video streams.
- ❑ Currently, there are 3 generally supported video formats for the **<video>** element:
 - MP4, WebM, and Ogg (MP4 is most commonly used, however WebM produces smaller files that load faster)
 - Not all formats are supported by all browsers

```
<video width="320" height="240" controls>  
  <source src="movie.mp4" type="video/mp4">  
  <source src="movie.ogg" type="video/ogg">  
  Your browser does not support the video tag.  
</video>
```



<source> Tag

- ❑ The **<source>** tag is used to specify multiple media resources for media elements, such as **<video>** and **<audio>**.
- ❑ The **<source>** tag allows you to specify alternate video/audio files which the browser may choose from, based on its media type or codec support.
- ❑ The browser will select the first source it supports.
- ❑ Attributes:
 - **src**= Specifies the URL of the media file
 - **type**= Specifies the MIME type of the media resource

```
<video width="320" height="240" controls>  
  <source src="movie.mp4" type="video/mp4">  
  <source src="movie.ogg" type="video/ogg">  
  Your browser does not support the video tag.  
</video>
```



Responsive Images

- ❑ There are two main considerations regarding responsive images on webpages:
 - ❑ Changing sizes/resolutions for different size screens to optimize bandwidth (it is slow and inefficient to load a large image for a small screen) – i.e., technical reasons.
 - ❑ Artistic choice – loading a different image for different size screens for design reasons.



Responsive Images Using the Tag

- ❑ Additional attributes in the tag are:
 - ❑ **srcset**= Specifies a comma-separated list of possible files to load, along with their original widths in pixels (not the intended display width) using the 'w' unit.
 - ❑ **sizes**= Specifies a comma-separated list of media conditions and the intended display width to use in either px or vw units. The last entry cannot have a condition, it is the default if nothing else applies.
- ❑ The browser will stop at the first match in **sizes**, and pick the image in **srcset** with a size matching the intended display size, or the first image with a larger size. (Note: browser may choose a different image depending on screen sharpness.)

```

```

```
img {  
  max-width: 100%;  
  height: auto  
}
```

A browser window 900px wide will match the second entry in **sizes**, calculate the intended display width of 450px and load the holiday_MED.png image.

<embed> Tag

- ❑ The `<embed>` tag defines a container for an external resource, such as a web page, an image, a video, etc.
- ❑ You shouldn't use the embed tag
 - Use `<img>` or `<picture>` to display a picture
 - Use `<iframe>` to display HTML
 - Use `<video>` and `<audio>` to display audio/video

```
<!-- Embedded image -->
<embed type="image/jpg" src="image.jpg" width="200" height="200">

<!-- Embedded HTML -->
<embed type="text/html" src="page.html" width="300" height="200">

<!-- Embedded Video -->
<embed type="video/webm" src="video.mp4" width="300" height="200">
```


Assuming you are designing a webpage to review an eBook. Which HTML element is most semantically appropriate to define a chapter review which also contains a single chapter of the eBook?

article

0%

chapter

0%

section

0%

div

0%

header

0%



HTML Forms

HTML Form Element

A form usually contains normal content and markup, plus “controls” for inputting data.

- ❑ Controls are buttons, text boxes, checkboxes, radio buttons, menus, ...
- ❑ Can also place labels on those controls.
- ❑ A specific button on the page is clicked to submit the data.

Height (cm):
Weight (kg):

Which disease(s) do you have?

- ☐ Exam-phobia
- ☐ Deadline-phobia
- ☐ Lecture-sleepiness

What is your profession?

Password:



HTML Form Element

Labels → Height (cm):
Weight (kg):

Textbox ←

Favourite Language

☐ HTML **Radio button** ←
☐ CSS

Which disease(s) do you have?

☐ Exam-phobia **Checkbox** ←
☐ Deadline-phobia
☐ Lecture-sleepiness

What is your profession? **Dropdown** ←

Textarea → Password:
Please put here any other health information.

Button →



<form> Tag

- ❑ The `<form>` element is used to create an HTML form
- ❑ The `<form>` element is a container for different input elements, such as text fields, radio boxes, checkboxes, drop down menus, date selectors, etc.

`<form>`

...

form elements

...

`</form>`

HTML Form Example

```
<form>  
  <label for="height">Height (cm):</label>  
  <input type="text" id="height"><br>  
  <label for="weight">Weight (kg):</label>  
  <input type="text" id="weight"><br>  
</form>
```

Height (cm):

Weight (kg):



<label> Tag

```
<label for="uname">Username:</label>  
<input type="email" id="uname"><br><br>
```

Username:

```
<label for="passwd">Password: </label>  
<input type="password" id="passwd"><br><br>
```

Password:

```
<label for="donation">How much do you want to donate?</label><br>  
$1<input type="range" id="donation" min="1" max="100">$100
```

How much do you want to donate?

\$1 \$100

Labels <label>

- ❑ Defines a label for input elements (and textarea, progress, etc)
- ❑ The **for** attribute of label must be equal to the **id** attribute of the corresponding input element.
- ❑ Useful for screen readers (will read out the label)
- ❑ When you click on a label for a checkbox/radio, the checkbox/radio will be toggled.



<input> Tag

- ❑ Most controls are specified by <input>
- ❑ `<input type="">`
- ❑ Possible options for type are
 - text | password | submit | reset
 - button | checkbox | radio | file
 - hidden | image | color | date |
 - month | time | week | search | range
 - | url | number | tel | email
- ❑ `<input>` is displayed inline by default, be careful you don't create a single-line form!

Common attributes

- ❑ **name**: the name of the form element
- ❑ **autocomplete**: either on or off
- ❑ **required**: an input must be provided before the form can be submitted
- ❑ **value**: (initial) value of the input element/**value sent to the server**
- ❑ **min/max**: min/max for date and number values

<input> Tag (type Attribute)

- ❑ **color** Defines a colour picker
- ❑ **date** Defines a date control (dd/mm/yyyy)
- ❑ **datetime** | **datetime-local** Defines a date and time control
- ❑ **month** Defines month and year control
- ❑ **week** Defines week and year control
- ❑ **time** Defines control for entering a time
- ❑ **search** Defines a text field for entering a search string
- ❑ **tel** Defines a field for entering a telephone number
- ❑ **number** Defines a field for entering number
- ❑ **range** Defines a slider control for entering a number
- ❑ **email** Defines a field for an e-mail address
- ❑ **url** Defines text field for entering a URL



<input> Tag (Radio Buttons)

Radio buttons <input type="radio">

- ❑ Let users **select only one option** from the available choices.
- ❑ Usually presented as a collection of radio buttons for a set of related options.
- ❑ All **input** tags in the **radio group** must have **the same value for the name attribute**.
- ❑ **value**: the value that is sent to the server when the form is submitted.

```
<input type="radio" name="urgency" id="low">
<label for="low">Low</label><br>

<input type="radio" name="urgency" id="medium">
<label for="medium">Medium</label><br>

<input type="radio" name="urgency" id="high">
<label for="high">High</label>
```

Urgency

☐ Low

☐ Medium

☐ High



<input> Tag (Checkboxes)

Checkboxes <input type="checkbox">

- ❑ Let users **select one or more options** from the available choices.
- ❑ Usually presented as a collection of checkbox buttons for a set of related options.
- ❑ All **input** tags in the checkbox group must have the same value for the **name** attribute.
- ❑ **value**: the value that is sent to the server when the form is submitted.

```
Which disease(s) do you have?<br>
<input
  type="checkbox"
  id="deadline"
  name="disease"
  value="deadline-fever"
  checked>
<label for="deadline">Deadline-phobia</label><br>
<input
  type="checkbox"
  id="lecture"
  name="disease"
  value="lecture-sleepiness">
<label for="lecture">Lecture-sleepiness</label><br><br>
```

Checked by default

Which disease(s) do you have?

☒ Deadline-phobia

☐ Lecture-sleepiness

<input> Tag (Pickers)

```
<label for="colour">Theme colour:</label>
<input type="color" id="colour"><br><br>

<label for="profilepic">Profile picture:</label>
<input
  type="file"
  id="profilepic"
  accept="image/png, image/jpeg"><br><br>

<label for="dob">Date of birth:</label>
<input type="date" id="dob">
```

Colour picker → Theme colour:

File picker → Profile picture: No file chosen

Date picker → Date of birth: 

August 2021 ▾ ↑ ↓

Su.	M.	Tu.	W.	Th.	F.	Sa.
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31	1	2	3	4
5	6	7	8	9	10	11

Today



<input> Tag (Submit & Reset)

<input type="submit">

- ❑ Defines a button for submitting form data to a form-handler

<input type="reset">

- ❑ Defines a button that will reset all form values to their default values.

```
<label for="height">Height (cm):</label>
<input type="text" id="height"><br><br>

<label for="weight">Weight (kg):</label>
<input type="text" id="weight"><br><br>

<input type="submit" value="submit">
<input type="reset">
```

Height (cm):

Weight (kg):



<button> Tag

<button>

- ❑ Defines a clickable button
- ❑ You can put content (text/image) inside a button
- ❑ You can also put tags like `<i>`, `<b>`, `<br>`, etc inside a button tag.
 - ❑ This is not possible with the `<input>` element!
- ❑ Wraps around markup to make the entire area act as a button

Note: *always specify the type attribute for a button element*

- ❑ **type = "submit | reset | button"**
 - ❑ Tells the browser which `<input>` type the button acts like

```
<button type="reset" value="reset">  
  <i>Reset</i>  
</button>  
<input type="submit" value="submit">
```





<textarea> Tag

- ❑ **<textarea>**
 - A multi-line text control
 - The initial value is the content of the tag
- ❑ **name = "name"**
- ❑ **cols = "number"**
 - Specifies the visible width (in characters)
- ❑ **rows = "number"**
 - Specifies the visible height (in characters)

```
<textarea name="info" rows="3" cols="40">  
    Please put here any other health information.  
</textarea>
```

Please put here any other health information.



Drop-Down Menus

<select>

- ❑ Container for a multiple-choice menu.
- ❑ Can only contain **<option>** tags
- ❑ **multiple = "multiple"**
 - Allows the user to select more than one option
- ❑ **size = "number"**
 - How many different options are visible at one time
 - **size="1"** acts as a drop down list
 - Higher values act as a scrolling lists

<option>

- ❑ One option within a **<select>** container
- ❑ The text between the begin and end tags is **displayed in the menu**
- ❑ **selected = "selected"**
 - Specifies if this option is initially selected
- ❑ **value = "text"**
 - The return value if this option is selected

Drop-Down Menus

```
<label for="profession">What is your profession?</label>
<select name="profession" id="profession">
  <option value="monster-killer">
    Monster Killer
  </option>
  <option value="alien-researcher">
    Alien Researcher
  </option>
  <option value="cartoon-director">
    Cartoon Director
  </option>
</select><br><br>
```

What is your profession? Monster Killer ▼

Password:

Monster Killer

Alien Researcher

Cartoon Director

Please put here any other



Grouping With the <fieldset> Tag

<fieldset>

- ❑ to group the different kinds of fields

<legend>

- ❑ The name of the group

```
<fieldset>
  <legend>Personalia:</legend>
  <label for="height">Height (cm):</label>
  <input type="text" id="height"><br><br>

  <label for="weight">Weight (kg):</label>
  <input type="text" id="weight"><br>
</fieldset>
```

Personalia:

Height (cm):

Weight (kg):



Form Action (Processing Form Input)

- ❑ Allows Web pages with simple interaction
 1. User **enters** data into form
 2. Browser "**submits**" form to **server**
 3. **Server** processes data and responds
- ❑ **Response**: update internal records, dynamically generate Web page, ...
- ❑ Processing on server performed by CGI scripts, or JSP/ASP pages, server supported scripts (such as Python, PHP, and Perl), server-side JavaScript, etc.
- ❑ **CGI** = Common Gateway Interface
- ❑ Defines how form data is passed to and from a server-side script for dynamic processing
- ❑ Other server-side scripting languages (JSP, ASP, ...) use similar mechanisms
- ❑ The output of most scripts is returned to the client, generally some HTML page



Forms and CGI

action = "url"

- ❑ The URL of the server page which will process the form

method = "get | post"

- ❑ The HTTP request used to submit the form to the server

get

- ❑ The form data is appended in the URL in name/value pairs
 - **`url?input1=value1&input2=value2&...`**
- ❑ Never use it to send sensitive data
- ❑ Size limitation (3000 Characters)
- ❑ Results can be bookmarked

post

- ❑ Form data is sent inside the body of HTTP request
- ❑ No size limitation
- ❑ Results can not be bookmarked

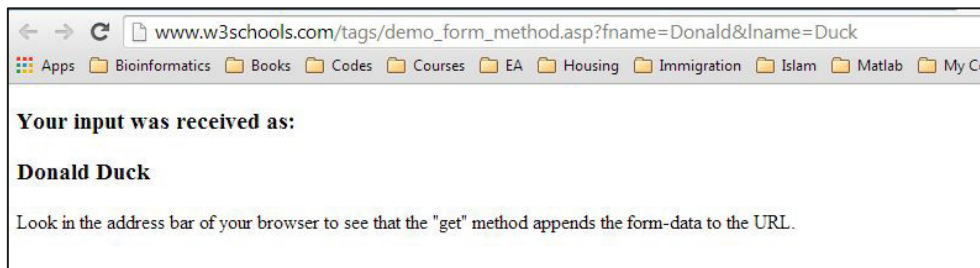


```
<form action="demo_form_method.asp" method="get">  
  First name: <input type="text" name="fname"><br>  
  Last name: <input type="text" name="lname"><br>  
  <input type="submit" value="Submit">  
</form>  
<p>  
  Click on the submit button, and the input will be sent to a  
  page on the server called "demo_form_method.asp".  
</p>
```

First name:

Last name:

Click on the submit button, and the input will be sent to a page on the server called "demo_form_method.asp".



```
<form action="http://www.cs.tut.fi/cgi bin/run/~jkorpela/echo.cgi" method="get">
```

Or email the form to your email

```
<form action="mailto:your@email.com" method="post">
```

Which of the following TWO statements are true for submitting forms using a GET request?

The submission is secure.

0%

Only a limited amount of data can be submitted.

0%

The result cannot be bookmarked.

0%

The data is sent in the URL rather than the request body.

0%

GET requests are more commonly used compared to POST requests for form submission.

0%



HTML Tables

HTML Tables

- ❑ Tables are used on web pages to organise tabular information
- ❑ Composed of rows and columns – similar to a spreadsheet.
- ❑ Each individual table cell is at the intersection of a specific row and column.
- ❑ Tables are created in HTML using the `<table>` tag

Name	Birthday	Phone
Jack	5/13	857-555-5555
Sparky	11/28	303-555-5555

Tables

A table can contain

- ❑ `<caption>...</caption>` – text to display as the table's **caption**
- ❑ `<thead>...</thead>` – rows to display as the table's **header**
- ❑ `<tfoot>...</tfoot>` – rows to display as the table's **footer**
- ❑ `<tbody>...</tbody>` – rows to display as the table's **body** (the content of the table)

Tables

- ❏ A **table row** is a tag `<tr>...</tr>`
- ❏ Each row contains possibly many
 - `<th>...</th>` – **table heading** cells
 - `<td>...</td>` – **table data** cells
- ❏ Given cells can “span” multiple rows or columns
 - **colspan** = number of columns spanned
 - **rowspan** = number of rows spanned
- ❏ Cells can contain any body tags
 - Even nested tables



HTML Table Example

```
<table border="1">
  <caption>Rainfall in Newcastle</caption>
  <thead>
    <tr>
      <th>Month</th>
      <th>Mean Rainfall (mm)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>January</td>
      <td>87.9</td>
    </tr>
    <tr>
      <td>February</td>
      <td>108.5</td>
    </tr>
    <tr>
      <td>March</td>
      <td>119.4</td>
    </tr>
    <tr>
      <td>Yearly Average</td>
      <td>94.3</td>
    </tr>
  </tbody>
</table>
```

HTML table with some added features

Rainfall in Newcastle	
Month	Mean Rainfall (mm)
January	87.9
February	108.5
March	119.4
Yearly Average	94.3



colspan, rowspan Attributes

```
<table border="1">
  <tr>
    <td colspan="3">
      Important Dates</td>
    </tr>
    <tr>
      <td rowspan="2">Birthday</td>
      <td>James</td>
      <td>11/08</td>
    </tr>
    <tr>
      <td>Karen</td>
      <td>4/17</td>
    </tr>
    <tr>
      <td rowspan="2">Anniversary</td>
      <td>James</td>
      <td>17/03</td>
    </tr>
    <tr>
      <td>Karen</td>
      <td>21/06</td>
    </tr>
</table>
```

Important Dates		
Birthday	James	11/08
	Karen	4/17
Anniversary	James	17/03
	Karen	21/06

```

<h3>Now let's make this prettier</h3>
<table border="0" style="table-layout:auto">
  <caption>Pretty Table</caption>
  <thead style="background-color:lightblue">
    <tr>
      <td colspan="3">Header for table</td>
    </tr>
  </thead>

  <tbody style="color:navy;background-color:lightgrey;">
    <tr>
      <th>Header</th>
      <td>Top-left cell</td>
      <td>Top-right cell</td>
    </tr>
    <tr>
      <th>Header</th>
      <td>Bottom-left cell</td>
      <td>Bottom-right cell</td>
    </tr>
    <tr>
      <td rowspan="2" style="background-color:yellow">
        Double<br>Cell
      </td>
      <td></td>
      <td></td>
    </tr>
    <tr>
      <td></td>
      <td></td>
    </tr>
  </tbody>

  <tfoot style="background-color:lightgreen">
    <tr>
      <td colspan="3">Some footer information</td>
    </tr>
  </tfoot>
</table>

```

Now let's make this prettier

Pretty Table

Header for table		
Header	Top-left cell	Top-right cell
Header	Bottom-left cell	Bottom-right cell
Double Cell		
Some footer information		

Tables and CSS

- ❑ **width** and **height** suggest the *minimum* size for a row or column

- but not necessarily the maximum

```
table { width: 100%; }  
th    { height: 50px; }
```

sets the width of the **table** to 100%, and
the height of the **th** elements to 50px

- ❑ **border**: used to specify table border.

```
table, th, td {  
  border: 1px solid black;  
}
```

specifies a black border for **table, th, and td** elements

- ❑ Changing the Width of columns, the **colgroup** element comes after the **caption**, but before **thead**:

```
<table>  
  <caption>My Table</caption>  
  <colgroup>  
    <col style="width:40%">  
    <col style="width:30%">  
    <col style="width:30%">  
  </colgroup>
```

The **col** tag is useful for assigning some style to all columns, instead of repeating the same for each **td** tag. Only the **border**, **background**, **width**, and **visibility** properties can be set on this tag.

Tables and CSS

- ❑ `empty-cells:`
 - `show` | `hide`
- ❑ `table-layout:`
 - `auto` – adjust the width of each column to accommodate the content of all cells
 - `fixed` – set the width of each column by the cells of the first row only
 - The main benefit of `table-layout: fixed` is that the table renders much faster

Peter	Griffin
Lois	

Peter	Griffin
Lois	

table-layout: auto

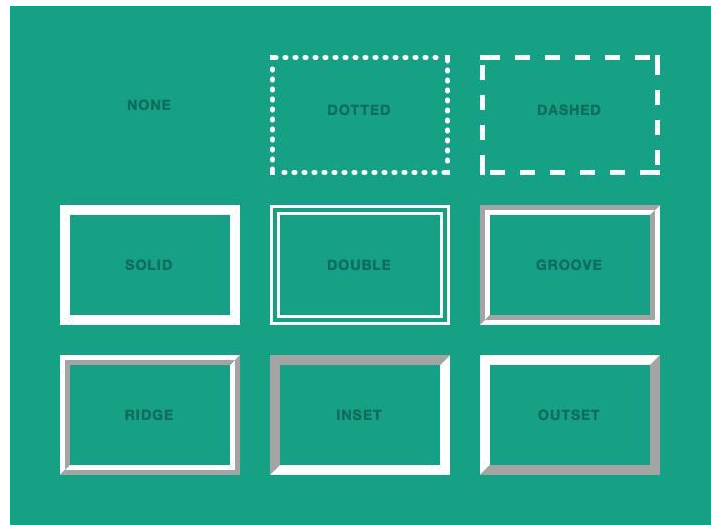
January 2021	87.9
February 2021	10000

table-layout: fixed

Jan 2021	87.9
February 2021	10000

Tables and CSS

- ❏ **border-style:**
 - As before, except...
 - **hidden = none** but **hidden** overrides adjacent styles
 - **inset/outset = ridge/groove** if **border-collapse** is set to **collapse** (see next slide)



https://tympanus.net/codrops/css_reference/border-style/

Tables and CSS

- ❑ All the tables we have seen so far have double borders.
 - Because the `table`, `th` and `td` elements have their own borders.
- ❑ To remove double borders, use **border-collapse** property.
- ❑ **border-collapse**: sets whether the table borders should be collapsed into a single border.

Jan 2021	87.4
Feb 2021	101.0

Jan 2021	87.4
Feb 2021	101.0

```
<table border="1" style="width:20%">
  <tbody>
    <tr>
      <td>Jan 2021</td>
      <td>87.4 </td>
    </tr>
    <tr>
      <td>Feb 2021</td>
      <td>101.0</td>
    </tr>
  </tbody>
</table>

<br>

<table border="1"
  style="width:20%; border-collapse: collapse">
  <tbody>
    <tr>
      <td>Jan 2021</td>
      <td>87.4 </td>
    </tr>
    <tr>
      <td>Feb 2021</td>
      <td>101.0</td>
    </tr>
  </tbody>
</table>
```



Tables and CSS

- ❑ You can use the **text-align** CSS property to define the horizontal alignment of the content in **<th>** and **<td>** elements.
- ❑ The **vertical-align** property sets the vertical alignment (top, bottom, middle) of the content in **<th>** and **<td>**
- ❑ These properties can be applied to either the entire table or rows/cells inside the table.

```
<table border="1" style="text-align:center">  
  <caption>Rainfall in Newcastle</caption>  
  <thead>  
    <tr>  
      <th>Month</th>  
      <th>Mean Rainfall (mm)</th>  
    </tr>  
  </thead>  
</table>
```

```
<tr style="height:50px; vertical-align:bottom">  
  <td>Yearly Average</td>  
  <td>94.3</td>  
</tr>
```

Rainfall in Newcastle

Month	Mean Rainfall (mm)
January	87.9
March	119.4
Yearly Average	94.3

CSS Reset



Removing styling from HTML

- ❑ Each component of the web should have a single role, HTML for structure and semantics, XML for data, CSS for styling, etc.
- ❑ The styles not specified in CSS still use browser defaults, resulting in a partial mix of semantics and styling. As browsers have different defaults for HTML elements in their internal stylesheets, this can cause our webpages to look different across different browsers.
- ❑ We want control over the styling of our website, to ensure consistent presentation and user experience across different platforms.
- ❑ To achieve this, we will need to specify a style for all HTML elements, to override every browser default.



CSS Reset

- ❑ A CSS reset is an external CSS file that is linked in the head element of an HTML file.
- ❑ It overrides all default browser styles, leaving most HTML elements looking identical. We can then add in our own CSS to style elements in a way that accomplishes our own design goals.
- ❑ There are many different CSS reset templates in common use, such as Eric Meyer's reset, Siolon's global reset, Tantek's reset, and many more. We will be using Eric Meyer's reset in this course.

My Favourite Things

HTML

Hypertext Markup Language (HTML) is the standard markup language for documents designed to be displayed in a *web browser*. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

- <https://en.wikipedia.org/wiki/HTML>
- https://upload.wikimedia.org/wikipedia/commons/thumb/6/61/HTML5_logo_and_wordmark.svg/180px-HTML5_logo_and_wordmark.svg.png

HTML is the Web's core language for creating content for everyone to use anywhere.



CSS Reset

- ❑ The CSS reset is usually the first external CSS linked, to ensure that our own styles will be applied on top of it.
- ❑ Applying a CSS reset:

<head>

<meta charset="UTF-8">

<title>CSS Reset</title>

<link rel="stylesheet" type="text/css" href="reset.css">

<link rel="stylesheet" type="text/css" href="style.css">

</head>

My Favourite Things

HTML

Hypertext Markup Language (HTML) is the standard markup language for documents designed to be displayed in a web browser. It can be assisted by technologies such as Cascading Style Sheets (CSS) and scripting languages such as JavaScript.

<https://en.wikipedia.org/wiki/HTML>

[https://upload.wikimedia.org/wikipedia/commons/thumb/6/61/HTML5_logo_and_wordmark.svg/180px-](https://upload.wikimedia.org/wikipedia/commons/thumb/6/61/HTML5_logo_and_wordmark.svg/180px-HTML5_logo_and_wordmark.svg.png)

[HTML5_logo_and_wordmark.svg.png](https://upload.wikimedia.org/wikipedia/commons/thumb/6/61/HTML5_logo_and_wordmark.svg/180px-HTML5_logo_and_wordmark.svg.png)

HTML is the Web's core language for creating content for everyone to use anywhere.

- ❑ The cascading order of CSS will cause the reset to be applied first, followed by our own external CSS file.

CSS Layout

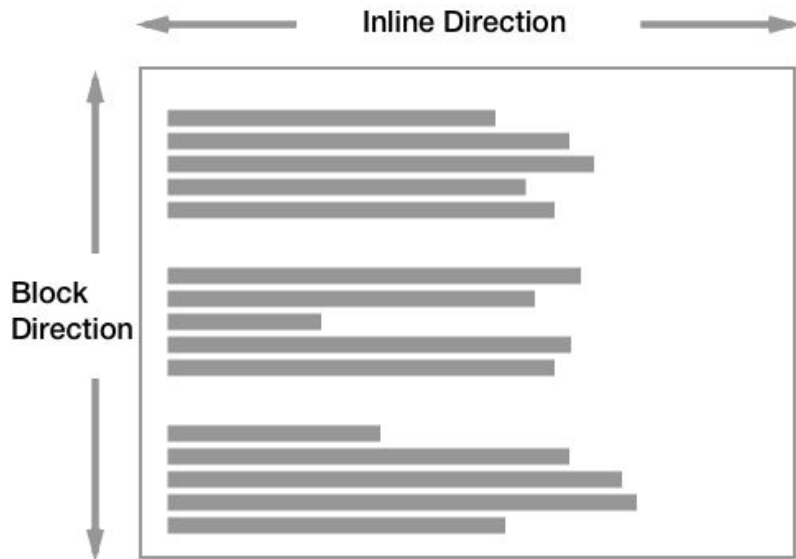


Normal Document Flow

Normal Flow

The default way in which elements are rendered and positioned, allowing for size and margins.

- ❑ In a **block formatting** context, boxes are laid out one after the other, vertically, beginning at the top of a containing block.
- ❑ In an **inline formatting** context, boxes are laid out horizontally, one after the other, beginning at the top of a containing block.



Source: https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Flow_Layout/Block_and_Inline_Layout_in_Normal_Flow



Element Display Types in Normal Flow

display: block

This will place the element in a block formatting context. The outer-left edge of the element will be aligned to the inner-left edge of the parent element. Separately, the outer-right edge will stretch to the inner-right edge of the parent element **regardless of width properties**, forcing non-child elements onto a newline. **Only vertical margins and padding will provide additional space between elements.**

display: inline-block

This will place the element in an inline formatting context, however width and height properties will be respected. Both horizontal and vertical margins and padding will provide additional space between elements.

display: inline

This will place the element in an inline formatting context. **Any height and width properties will be ignored**, being determined entirely by the content of the element. **Only horizontal margins and padding will provide additional space between elements.**

Element Display Types in Normal Flow

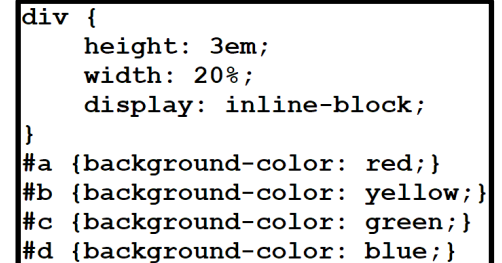


```
div {  
  height: 3em;  
  width: 20%;  
  display: block;  
}  
#a {background-color: red;}  
#b {background-color: yellow;}  
#c {background-color: green;}  
#d {background-color: blue;}
```



```
div {  
  height: 3em;  
  width: 20%;  
  display: inline;  
}  
#a {background-color: red;}  
#b {background-color: yellow;}  
#c {background-color: green;}  
#d {background-color: blue;}
```

```
<body>  
  
  <div id="a">A</div>  
  <div id="b">B<div id="c">C</div></div>  
  <div id="d">D</div>  
  
</body>
```



```
div {  
  height: 3em;  
  width: 20%;  
  display: inline-block;  
}  
#a {background-color: red;}  
#b {background-color: yellow;}  
#c {background-color: green;}  
#d {background-color: blue;}
```





CSS Positioning

Static Positioning

- ❑ The default that every element gets.
- ❑ Put the element into its normal position in the document flow.

```
<p class="positioned"> ... </p>
```

```
.positioned {  
  position: static;  
  background: yellow;  
}
```



CSS Positioning

Relative Positioning

- ❑ Once the positioned element has taken its place in the normal flow, you can then modify its final position, including making it overlap other elements on the page. This works because the element is still 'virtually' located in its original position in the normal flow.
- ❑ top, bottom, left, and right are used alongside position to specify exactly where to move the positioned element to.

```
<p class="rel"> ... </p>
```

```
.rel {  
  position: relative;  
  top: -50px;  
  left: -150px;  
}
```

This text is relatively positioned.

HTML

```
<div>  
  
<p> This is the  
normal... <span  
class="rel"> This  
text is relatively  
positioned. </span>  
... from top to  
bottom...</p>  
  
</div>
```

CSS

```
.rel {position: relative;  
      top: -50px;  
      left: -150px}
```

Relative

This is the normal flow of a document;
from top to bottom, left to right. When you add the relatively positioned text, it moves to the coordinates you set based on the top left corner of the containing element, in this case the <div>. Normal text flows as normal, but a gap is left where the relative text used to be, and the text overlaps with the newly positioned relative text if they are in the same area.



CSS Positioning

Absolute Positioning

- ❑ An absolutely positioned element no longer exists in the normal document flow. This also means that its parent will not expand to fit it.
 - ❑ Instead, it sits on its own layer separate from everything else. This means that elements in the normal flow may exist in the same space as the absolutely positioned element.
 - ❑ This is very useful: it means that we can create isolated UI features that don't interfere with the layout of other elements on the page.
 - Such as popup information boxes
- ❑ top, bottom, left, and right will position the element relative to its nearest parent that has a non-static position value. If none exist, then the root element is used.

```
<p class="abs"> ... </p>
```

```
.abs {  
  position: absolute;  
  top: 40px;  
  left: 80px;  
}
```

HTML

```
<div>

<p> This is the
normal... <span
class="abs"> This
text is absolutely
positioned.</span>
...top to bottom...
</p>

</div>
```

CSS

```
.abs {position:absolute;
      top: 40px;
      left: 80px;}
```

Absolute

This is the normal flow of a document; from top to bottom, left to right. When you add the absolutely positioned text, it moves to the coordinates you set based on the top left corner of the containing element, in this case the <div>. Normal text flows over the absolutely positioned text. There is no gap where the text is taken from.



CSS Positioning

Fixed Positioning

- ❑ An element with `position: fixed;` is positioned relative to the viewport
 - meaning it always stays in the same place even if the page is scrolled.
- ❑ The `top`, `right`, `bottom`, and `left` properties are used to position the element.
- ❑ Can create useful UI items that are fixed in place
 - persistent navigation menus that are always visible no matter how much the page scrolls.

```
<p class="fixed"> ... </p>
```

```
.fixed {  
  position: fixed;  
  top: 0px;  
  left: 10px;  
}
```

- ❑ The element is also removed from the normal flow, in fact the only major difference between absolute and fixed is that the position for fixed elements is always relative to the viewport.

The '**Fixed positioning**' header has a **position: fixed** CSS property.

Even after the elements after it have scrolled out of the viewport, it is still in the same position.



Fixed positioning

I am a basic block level element. My adjacent elements sit on new lines below me.

I'm not positioned any more...

We are separated by our margins. Because of collapsing, we are separated by the width margins, not both.

Fixed positioning

elements sit on new lines below me.

I'm not positioned any more...

We are separated by our margins. Because of collapsing, we are separated by the width margins, not both.

CSS Positioning

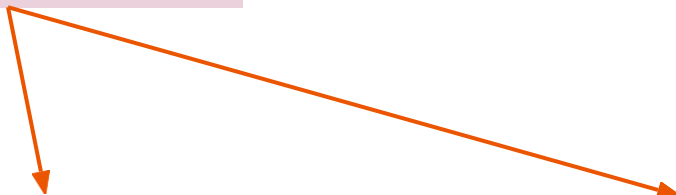
Sticky Positioning

- ❑ A hybrid between relative and fixed position.
- ❑ Allows a positioned element to act like it's relatively positioned until it's scrolled to a certain threshold relative to its nearest scrolling ancestor (in many cases, this is just the viewport), such as 10px from the top of the viewport.
- ❑ Then it becomes fixed.
- ❑ Example, cause a navigation bar to scroll with the page until a certain point and then stick to the top of the page.

```
<p class="sticky"> ... </p>
```

```
.sticky {  
  position: sticky;  
  top: 30px;  
  left: 30px;  
}
```

Can scroll to the specified position (top, in this case)



voluptat. suspendisse ac imperdiet turpis. Aenean
sollicitudin eros pharetra congue. Duis ornare
luctus. Proin blandit quam nec lacus varius con
Ut id ornare felis, eget fermentum sapien.

Sticky

Nam vulputate diam nec tempor bibendum. I

Sticky

Nam vulputate diam nec tempor bibendum
eget malesuada ultrices. Phasellus turpis
dapibus ut, facilisis sed est. Nam id risus
consectetur eget aliquam lorem. Vivamus

sociis natoque penatibus et magnis dis pa
nascetur ridiculus mus.

Sticky

Lorem ipsum dolor sit amet, consectetur a
luctus aliquam dolor, eu lacinia lorem pla
felis orci, pulvinar id metus ut, rutrum lue
imperdiet nunc, at ultricies tellus laoreet
cursus massa at porta. Integer ligula ipsu
vel, viverra egestas ligula. Curabitur vehi

Once there, it sticks to its position.
Doesn't scroll away



Float Property

- ❑ The **float** property is used for positioning and formatting content
 - ❑ e.g. let an image float left to the text in a container.
- ❑ The **float** property can have one of the following values:
 - ❑ **left**: float to the left of its container
 - ❑ **right**: float to the right of its container
 - ❑ **none**: does not float
- ❑ Only text/inline elements will wrap around a float. Block elements will be positioned as though the floated element did not exist. This can become **very** confusing.

```
<p class="float"> ... </p>
```

```
.float {  
  float: left;  
}
```

- ❑ Float works by calculating the element's position in the normal flow, then taking the element **out of the normal flow** and moving it in the direction specified by the float until the edge of the parent element is reached, or the edge of another floated element.

HTML

```
<div>  
  
<p> This is the  
normal...</p>  
  
<p  
class="float">This  
s text is floated  
right.</p>  
  
</div>
```

CSS

```
.float  
{float:right;}
```

Float

This is the normal flow of a document; from top to bottom, left to right. When the floated text is added, it moves to the top right corner of the containing element, in this case the `<div>`. Normal text flows around the floated text.

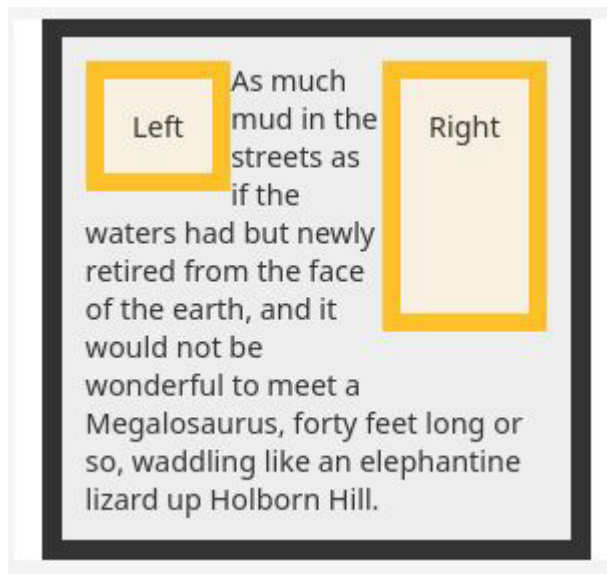
**This text is
floated
right.**



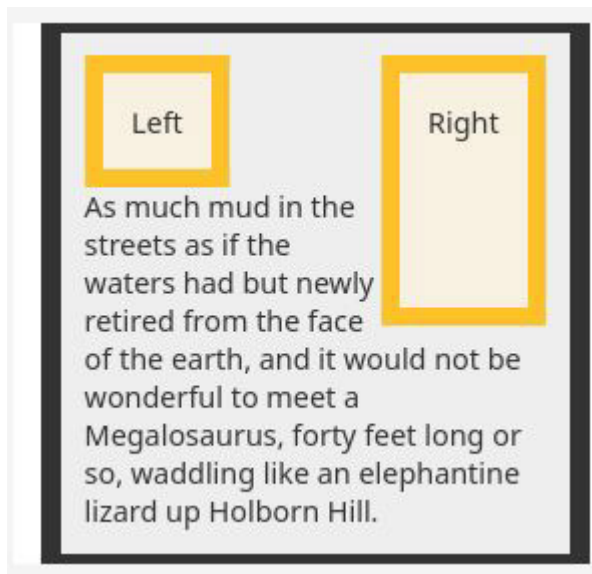
Clear Property

- ❑ The CSS **clear** property controls the flow next to floated elements (push away from specified float).
- ❑ It specifies what should happen to the flow element that is next to a floating element.
 - Whether and how it should be moved below floating elements
- ❑ Possible options are
 - **none**: the flow element is not pushed below left or right floated elements
 - **left**: the flow element is pushed below left floated elements
 - **right**: the flow element is pushed below right floated elements
 - **both**: the flow element is pushed below both left and right floated elements

```
clear: none;
```



```
clear: left;
```



Source: <https://developer.mozilla.org/en-US/docs/Web/CSS/clear>

`clear: right;`

Left

Right

As much mud in the streets as if the waters had but newly retired from the face of the earth, and it would not be wonderful to meet a Megalosaurus, forty feet long or so, waddling like an elephantine lizard up Holborn Hill.

`clear: both;`

Left

Right

As much mud in the streets as if the waters had but newly retired from the face of the earth, and it would not be wonderful to meet a Megalosaurus, forty feet long or so, waddling like an elephantine lizard up Holborn Hill.

Source: <https://developer.mozilla.org/en-US/docs/Web/CSS/clear>

Overflow Property

- ❑ The CSS **overflow** property sets the desired behaviour for when the content of an element overflows
 - When the content is too big to fit in the element's block.
 - Considers both directions: horizontal and vertical
- ❑ Possible options are
 - **visible**: content is not clipped and might be visible beyond the box of the element.
 - **scroll**: browsers always display scrollbars whether or not any content is actually clipped
 - **hidden**: content is clipped if necessary to fit within the element's padding box. If the element only contains floated elements, it will expand to fit them.
 - Scroll bars are hidden
 - Allows programmatic scrolling
 - **clip**: same as hidden, but doesn't allow scrolling.
 - **auto**: the browser decides what best to do - depending on whether the content overflows

`overflow: visible;`

`overflow: hidden;`

`overflow: scroll;`

Michaelmas term lately
over, and the Lord
Chancellor sitting in
Lincoln's Inn Hall.
Implacable November
weather. As much mud in
the streets as if the waters
had but newly retired from
the face of the earth.

Michaelmas term lately
over, and the Lord
Chancellor sitting in
Lincoln's Inn Hall.
Implacable November

Chancellor sitting in
Lincoln's Inn Hall.
Implacable November
weather. As much mud in
the streets as if the waters
had but newly retired from

Columnar Layouts

- ❑ The CSS `column-count` and `column-width` properties allow for a block element to be displayed in a columnar layout.
- ❑ The shorthand `columns` can be used to set both properties at once:

```
columns: 3 auto;
```

Lorem ipsum dolor sit
amet, consectetur
adipiscing elit, sed do
eiusmod tempor
incididunt ut labore et
dolore magna aliqua. Ut
enim ad minim veniam,

quis nostrud exercitation
ullamco laboris nisi ut
aliquip ex ea commodo
consequat. Duis aute irure
dolor in reprehenderit in
voluptate velit esse
cillum dolore eu fugiat

nulla pariatur. Excepteur
sint occaecat cupidatat
non proident, sunt in
culpa qui officia deserunt
mollit anim id est
laborum.



Flexbox

`display: flex`

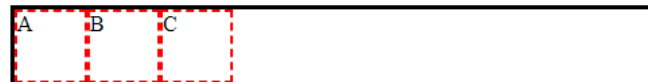
- ❑ This will place the element in a flexible box layout, which behaves like a block-level element, but allows the child elements to be laid out very precisely, while also allowing them to grow and shrink with the size of the viewport. The important properties for positioning content are:
- ❑ `justify-content=flex-start|flex-end|center|space-between|space-around|space-evenly` specifies how horizontal space should be distributed between child items.
- ❑ `align-items=flex-start|flex-end|center` specifies how items should be positioned vertically.
- ❑ `flex-direction=row|row-reverse|column|column-reverse` specified how items are laid out in a flexbox. If a column direction is set, then the horizontal and vertical axis are swapped.

```
<div id="parent">
  <div class="child">A</div>
  <div class="child">B</div>
  <div class="child">C</div>
</div>
```

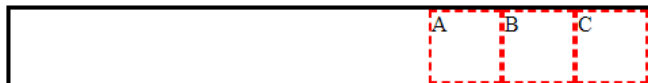
```
#parent {
  display: flex;
  border: 3px solid black;
}

.child {
  width: 50px;
  height: 50px;
  border: 2px dashed red;
}
```

```
#parent {justify-content: flex-start;}
```



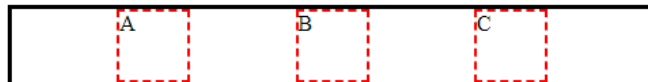
```
#parent {justify-content: flex-end;}
```



```
#parent {justify-content: space-around;}
```



```
#parent {justify-content: space-evenly;}
```



```
#parent {justify-content: space-between;}
```



CSS Flexbox

- ❑ Flexbox has additional properties that control how individual items grow and shrink with the viewport:
- ❑ **flex-basis**= sets the initial size for a child item of a flexbox
- ❑ **flex-grow**= controls how items in a flexbox will expand, the value for each item is calculated as a ratio so that items grow proportionally to one another.
- ❑ **flex-shrink**= controls how items in a flexbox will shrink, this is also calculated as a ratio for each item as with **flex-grow**.
- ❑ **order**= allows the order of individual items within a flexbox to be changed from the default layout.

```
<div id="parent">
  <div class="child" style="order: 4">A</div>
  <div class="child" style="order: -1">B</div>
  <div class="child">C</div>
</div>
```

```
#parent {
  display: flex;
  border: 3px solid black;
}

.child {
  width: 50px;
  height: 50px;
  border: 2px dashed red;
}

#parent {justify-content: space-around;}
```

By default, each element has an order of zero.

Elements are positioned increasingly according to the value of their order attribute.

If multiple elements have the same order value, the order of the elements in the HTML is used.



Which TWO of the following position values will NEVER remove an element from the normal flow?

absolute

0%

relative

0%

fixed

0%

static

0%

sticky

0%





Summary

- ❑ We have finally completed the separation of styling and structure, using a CSS reset to clear the browser defaults. This gives us full control over the styling of our webpages and firmly defines the role of HTML as specifying only structure and semantics.
- ❑ We have also learned to use CSS to create full page layouts, using positioning, floats, and display types such as flexbox.
- ❑ Finally, we learned to use some more advanced features of HTML such as tables and forms to display complex data and receive input from the user.
- ❑ But there's now a new problem: what happens when the user submits a form on our website? And for that matter, up until now we've only been creating webpages on our own computer; how do we let other people view our website? It's time to look at moving our site to a web server, find out how web servers respond to requests and form submissions, and learn how we can create our own behaviour – next week.

Summary

- ❑ What are the semantics structural elements in HTML5
 - `<header>`, `<footer>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`,
 - `<details>`, `<figure>`, `<figcaption>`, `<mark>`, `<progress>`, `<summary>`, `<time>`
- ❑ HTML5 `<audio>`, `<video>` elements
- ❑ HTML5 `<form>` element
 - Different types of inputs in form element
 - Methods for submitting a form
 - New attributes in `<form>` element
 - New input types for `<form>` element

Summary

- ❑ How to use tables to organize your data
- ❑ What are the components of HTML table: **thead, tbody, tr, td, th, tfoot, caption**
- ❑ How to design complex tables: **rowspan, colspan**
- ❑ How CSS can be used to make tables more interesting and exciting
- ❑ CSS provides different “positioning schemes” to control the placement of blocks: fixed, relative, absolute

References

- ❏ **Web Development and Design Foundations with HTML5 (6e)**

By Terry Felke-Morris

- ❏ *Chapter 6: Section 6.11*

- ❏ *Chapter 8, 9*

- ❏ http://www.w3schools.com/html/html5_intro.asp